

Article

A Deep Reinforcement Learning-Driven Seagull Optimization Algorithm for Solving Multi-UAV Task Allocation Problem in Plateau Ecological Restoration

Lijing Qin *, Zhao Zhou, Huan Liu , Zhengang Yan and Yongqiang Dai

College of Information Science and Technology, Gansu Agricultural University, Lanzhou 730070, China; 1073323120805@st.gsau.edu.cn (Z.Z.); liuh@gsau.edu.cn (H.L.); yanzhg@gsau.edu.cn (Z.Y.); daiyq@gsau.edu.cn (Y.D.)

* Correspondence: qinlij@gsau.edu.cn

Abstract: The rapid advancement of unmanned aerial vehicle (UAV) technology has enabled the coordinated operation of multi-UAV systems, offering significant applications in agriculture, logistics, environmental monitoring, and disaster relief. In agriculture, UAVs are widely utilized for tasks such as ecological restoration, crop monitoring, and fertilization, providing efficient and cost-effective solutions for improved productivity and sustainability. This study addresses the collaborative task allocation problem for multi-UAV systems, using ecological grassland restoration as a case study. A multi-objective, multi-constraint collaborative task allocation problem (MOMCCTAP) model was developed, incorporating constraints such as UAV collaboration, task completion priorities, and maximum range restrictions. The optimization objectives include minimizing the maximum task completion time for any UAV and minimizing the total time for all UAVs. To solve this model, a deep reinforcement learning-based seagull optimization algorithm (DRL-SOA) is proposed, which integrates deep reinforcement learning with the seagull optimization algorithm (SOA) for adaptive optimization. The algorithm improves both global and local search capabilities by optimizing key phases of seagull migration, attack, and post-attack refinement. Evaluation against five advanced swarm intelligence algorithms demonstrates that the DRL-SOA outperforms the alternatives in convergence speed and solution diversity, validating its efficacy for solving the MOMCCTAP.

Keywords: multi-UAV; collaborative task allocation; deep reinforcement learning; seagull optimization algorithm



Academic Editor: Pablo Rodríguez-González

Received: 14 April 2025

Revised: 2 June 2025

Accepted: 12 June 2025

Published: 14 June 2025

Citation: Qin, L.; Zhou, Z.; Liu, H.; Yan, Z.; Dai, Y. A Deep Reinforcement Learning-Driven Seagull Optimization Algorithm for Solving Multi-UAV Task Allocation Problem in Plateau Ecological Restoration. *Drones* **2025**, *9*, 436. <https://doi.org/10.3390/drones9060436>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

With the rapid development of UAV technology, its mobility, flexibility, and lower risk characteristics have made it an indispensable and important tool in many fields, and it is widely used in agricultural monitoring, military reconnaissance, emergency rescue, and other scenarios [1,2]. Among them, agricultural UAVs, as important equipment for modern agriculture, have played a key role in tasks such as aerial monitoring, precision irrigation, plant protection spraying, and crop sowing, which not only can effectively replace manpower, but also significantly improve the efficiency of agricultural production. However, with the increasing complexity of agricultural production needs, the limitations of a single agricultural UAV in terms of resource-carrying capacity and task execution capacity are becoming more and more obvious, and it is difficult to independently cope with diversified farmland management tasks. In this context, the advantages of a system

with multiple agricultural UAVs are gradually emerging. The system has higher flexibility and fault tolerance and can efficiently accomplish tasks in more complex agricultural production environments. However, the simple integration of multiple agricultural UAVs into clusters fails to fully utilize their potential advantages. In the absence of a perfect synergy mechanism, it is not only difficult to improve the execution efficiency of agricultural production tasks, but also may lead to serious task conflicts and resource wastage due to the expansion of scale [3]. Therefore, in complex agricultural scenarios with multiple constraints, how to reasonably allocate agricultural production tasks becomes the key to improving the overall performance of the agricultural production system, and is also the basis for realizing the cooperative operation of multiple agricultural UAVs.

The multi-UAV collaborative task allocation problem is the process of efficiently allocating and coordinating tasks in a system with multiple UAVs. The process aims to maximize the overall efficiency and performance of the system, ensure that the tasks can be completed on time, and make full use of the resources and capabilities of each UAV [4]. However, the problem has high complexity and constraints, so it is the focus of the research to reasonably model the specific practical application scenarios and design excellent solution algorithms for the problem model. At present, many scholars in this field have proposed numerous intelligent optimization algorithms to solve this problem. Traditional intelligent optimization algorithms usually rely on fixed heuristic rules and predefined search strategies, which may perform inconsistently in different problems or search stages and lack the ability to automatically adjust strategies based on actual problems. Therefore, an algorithm with excellent performance is proposed to solve the multi-UAV collaborative task allocation problem. The specific main contributions are as follows:

(1) A corresponding MOMCCTAP model was constructed for the practical scenario in which UAVs are required to restore the grassland ecology in a certain area, incorporating practical agricultural constraints such as UAV flight range, task priority, and multi-UAV coordination. The model optimizes two objectives: minimizing the maximum completion time for UAVs to complete and return, representing system efficiency, and minimizing the total flow time for all UAVs, reflecting overall resource consumption.

(2) A DRL-SOA is proposed to solve the MOMCCTAP model, where SOA serves as the environment for DRL. The framework incorporates a DQNAgent across three stages: migration, attack, and post-attack optimization. In the migration stage, the DQNAgent adjusts the search factor to balance global and local searches. During the attack stage, multiple local search strategies are introduced to diversify the search space. In the post-attack stage, the DQNAgent selects strategies to refine the task allocation. A reward–punishment mechanism guides the DQNAgent to optimize solutions adaptively, enhancing efficiency and effectiveness.

(3) Several sets of simulation experiments were designed to prove that the pareto front (PF) obtained by the proposed DRL-SOA is superior to the other five advanced heuristic algorithms and can solve such problems more effectively.

This paper is organized as follows: Section 2 provides a systematic review of the current research status of multi-UAV task allocation problems at home and abroad. Section 3 constructs a model of the MOMCCTAP for specific application scenarios. Section 4 elaborates on the DRL-SOA. Section 5 discusses the design and implementation of multiple sets of simulation experiments to verify the performance advantages of the DRL-SOA. Section 6 summarizes the main work of the full text and looks ahead to future research directions.

2. Related Work

To model real-world application scenarios, numerous multi-UAV cooperative task allocation models have been developed. For instance, Hu et al. [5] formulated a multi-target

tracking task allocation model by incorporating three key performance metrics—total tracking distance, fairness in task distribution, and timeliness of execution—under task-specific constraints. Wu et al. [6] proposed a UAV load-balancing optimization method by jointly considering flight distance and task execution time. Their evaluation framework quantitatively analyzed the overall task execution efficiency and coordination by measuring the cumulative flight path lengths of all UAVs and the total time required to complete tasks within the mission area. Ma et al. [7] addressed time constraints and the heterogeneous capabilities and task demands among UAVs, designing a model that reflects practical diversity in UAV systems. Liu et al. [8] introduced a two-stage centralized task assignment approach for forest resource reconnaissance, integrating expectation-maximization clustering with a multi-dimensional knapsack problem (KNKP) framework. Zhang et al. [9] accounted for multiple complex constraints from real-world mission environments and developed a UAV formation task planning model aimed at minimizing both task execution and route planning costs. Lastly, Peng et al. [10] presented a dynamic task allocation strategy inspired by the division of labor in wolf packs. Their method combined decentralized individual choices with centralized group decisions, enhancing the adaptability, coordination, and robustness of UAV swarm operations in dynamic environments.

In recent years, various algorithms have been developed to address multi-UAV task allocation and other combinatorial optimization problems, leveraging swarm intelligence for its robustness and adaptability. Wang et al. [11] applied multi-objective optimization and improved quantum behavior particle swarm optimization to enhance efficiency and accuracy. Zhou et al. [12] developed a Tent-Lévy-based seagull optimization algorithm (TLISOA) to address the multi-UAV task allocation problem in agricultural contexts. The proposed algorithm incorporates Tent chaotic mapping, the Lévy flight mechanism, and an adaptive spiral coefficient. This integration aims to enhance both the convergence rate and the solution precision, thereby improving the efficiency of UAV coordination in agricultural task execution. Liu et al. [13] proposed the MODRL-SIA algorithm, combining deep reinforcement learning and swarm intelligence to address location-routing problems with time windows, optimizing costs, carbon emissions, and product losses efficiently. Chen et al. [14] proposed a multi-objective ant colony optimization (MOACO) algorithm, improving convergence, solution quality, and diversity. Gao et al. [15] introduced an improved genetic algorithm with specialized operators for heterogeneous UAVs. Liu et al. [16] proposed a two-stage hybrid flow shop scheduling model that incorporates sequence-dependent setup times, and introduced an EDA-MIS algorithm to solve it effectively. The algorithm integrates heuristic initialization, probabilistic modeling, and neighborhood search to enhance scheduling efficiency and accuracy, with strong performance in large-scale problems. Xu et al. [17] utilized a shuffled frog-leap algorithm (MOSFLA) and genetic algorithm (GA) for UAV plant protection tasks, while Xu et al. [18] designed a differential evolution algorithm for battlefield task allocation with tailored operations. Long et al. [19] improved discrete pigeon optimization for unbalanced reconnaissance tasks, while Chen et al. [20] combined heuristic algorithms with coalition formation games for cooperative target assignment. Liu et al. [21] proposed a model for the Multi-Depot Capacitated Vehicle Routing Problem with Time Windows (MDCVRPTW) and developed the ALNSWWO algorithm. This algorithm combines adaptive neighborhood search strategies to optimize delivery costs and efficiency, showing superior performance in fresh agricultural product logistics. Chen et al. [22] developed a chaotic wolf pack algorithm integrating Gaussian random walks and adaptive mechanisms, effectively addressing heterogeneous UAV task allocation. Zhang et al. [23] proposed a dynamic task allocation approach using particle swarm optimization with a market auction mechanism. Yan et al. [24] introduced a modified genetic algorithm for cooperative attack assignments, incorporating customized crossover and mutation opera-

tions. Liu et al. [25] introduced the KbP-LaF-CMAES algorithm tailored for multimodal optimization problems. This approach integrates a knowledge-based perturbation (KbP) mechanism and a leaders-and-followers (LaF) strategy into an enhanced Covariance Matrix Adaptation Evolution Strategy (CMA-ES) framework. The approach balances exploration and exploitation, achieving superior convergence speed and accuracy at identifying multiple global optima. Yu et al. [26] employed an improved firefly algorithm incorporating chaotic mapping and Lévy flight mechanisms to enhance optimization performance in uncertain environments. These algorithms have also been adapted for broader combinatorial optimization problems, improving efficiency, convergence, and robustness in diverse scenarios. Liu et al. [27] proposed a DHFSP-DE model for aluminum production and developed an MTMA algorithm using evolutionary transfer learning to enhance scheduling efficiency. Experiments show its superiority over existing methods. In summary, most scholars focus on heuristic algorithms for fixed strategies, and these methods rely on pre-set heuristic rules and fixed search strategies. In contrast, the DRL-SOA proposed in this paper can dynamically and adaptively adjust the optimization strategy during the task allocation process through deep reinforcement learning. By embedding the DQNAgent in different stages of the seagull optimization algorithm, the DRL-SOA can autonomously learn better search strategies during training, rather than relying on pre-set rules, and is more adaptable.

3. Problem Formulation

3.1. Scenario Description

The grassland ecosystem plays a crucial role in agricultural production and regional ecological balance. However, in plateau regions, these ecosystems are facing severe challenges. Frequent marmot activities have led to significant vegetation destruction, soil degradation, and erosion, which not only undermine the stability of the grassland ecosystem but also reduce forage yields. This degradation threatens the sustainable development of livestock farming and the livelihoods of local communities. Additionally, such ecological damage can trigger a cascade of environmental repercussions, further impacting regional climate, wildlife habitats, and hydrological cycles, thereby posing a serious threat to the ecological equilibrium of plateau areas. In order to solve these problems and restore the ecological functions of the grasslands, multiple drones are now being deployed to perform multiple ecological restoration tasks in the grasslands of a certain plateau area. Suppose that in the highland area $D = [0, w] \times [0, w]$, n UAVs need to perform ecological restoration tasks at m task sites, and the UAVs that perform the tasks depart from the same command center at the same time, passing through and completing their assigned ecological restoration tasks in order and returning to the command center. Each ecological restoration task requires a different execution time and priority. The goal of cooperative task allocation is to complete all the ecological restoration tasks in the grassland in this area while satisfying some practical constraints.

Let the set of UAVs be $UAV = \{U_1, U_2, \dots, U_i, \dots, U_n\}$, the initial coordinates of the UAVs be $P = (X, Y)$, the flight speed of the UAVs be V , the maximum range of the UAVs be max_dis , the set of tasks to be performed by all the UAVs be $Task = \{Task_1, Task_2, \dots, Task_j, \dots, Task_m\}$, the positional coordinates of each task to be abstracted into be $Q_m = (o, q)$, the set of execution durations of the tasks be $Time = \{Time_1, Time_2, \dots, Time_j, \dots, Time_m\}$, the priority of the tasks in the sequence of assignments of the UAVs U_i be $p_{U_i} = \{p_1, p_2, \dots, p_g, \dots, p_u, \dots, p_{N_i}\}$, and the sequence of tasks performed by the UAVs U_i be denoted by $Seq^i = \{M_1^i, M_2^i, \dots, M_e^i, \dots, M_h^i, \dots, M_{N_i}^i\}$. The UAVs U_i perform a sequence of the tasks to which it has been assigned, and its flight distance is dis_i ; N_i denotes the number of tasks assigned to the UAV U_i . Define C_{ij} as a

decision variable that represents the assignment relationship between the UAV U_i and the task $Task_j$. When a task $Task_j$ is assigned to a UAV U_i , $C_{ij} = 1$; otherwise $C_{ij} = 0$. The schematic diagram is shown in Figure 1.

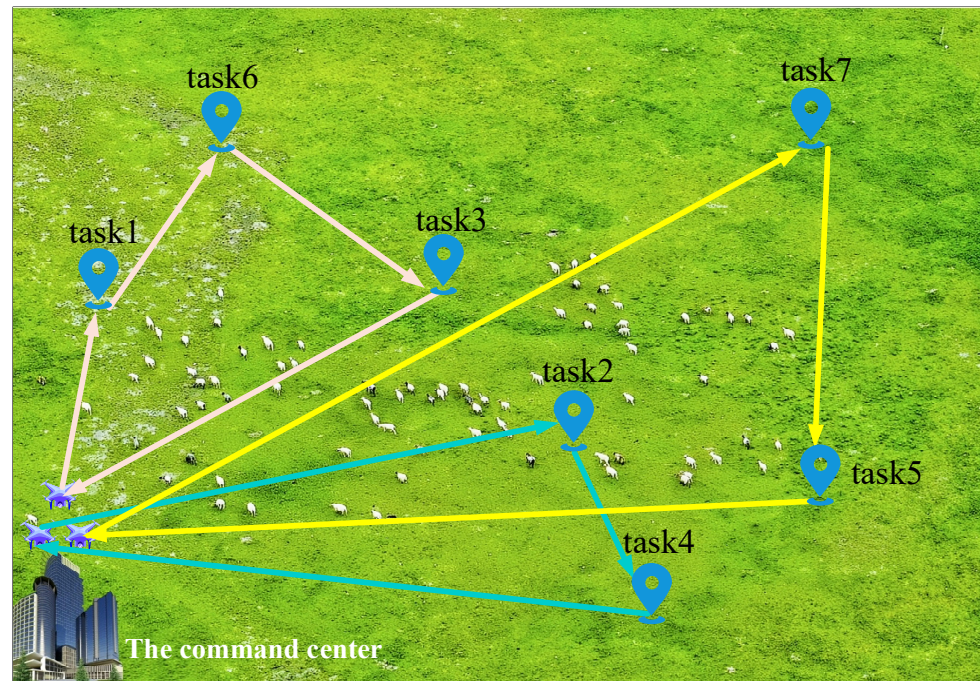


Figure 1. Schematic of problem model.

3.2. Restrictive Conditions

The following constraints exist in this task allocation problem:

(1) Multi-UAV collaboration constraint: Any task point in the set of task points for ecological restoration can only be accomplished once, i.e., all UAVs can only operate once on the same task point, i.e.,

$$Seq^a \cap Seq^b = \emptyset \quad a, b \in \{1, 2, \dots, n\} \text{ and } a \neq b \quad (1)$$

(2) Task completion constraint: all task points for ecological restoration must be completed, i.e.,

$$Seq^1 \cup Seq^2 \cup \dots \cup Seq^n = \{Task_1, Task_2, \dots, Task_j, \dots, Task_m\} \quad (2)$$

(3) Task priority constraint: in the execution sequence of a UAV task, tasks with higher task priorities are assigned first, i.e.,

$$\text{if } p_g < p_u, \text{ then } e > h \quad (3)$$

(4) Maximum range restriction for UAVs: The flight range of each UAV must not exceed its maximum range. That is,

$$dis_i \leq max_dis \quad (4)$$

3.3. Objective Functions

In order to make the UAV task assignment scheme more scientific and reasonable, the MOMCCTAP model established in this paper comprehensively considers two optimization objectives: the maximum completion time required for each UAV to complete all tasks

and return to the starting point, and the total flow time taken by all the UAVs to finish their assigned tasks and return to the starting point. The maximum completion time required for each UAV to complete all tasks and return to the starting point reflects the rate at which the entire UAV system completes all tasks, that is, the time it takes for the last UAV in the entire UAV system to complete the ecological restoration task and return to the command center. The sequence of tasks that UAV U_i performs in sequence is $Seq^i = \{M_1^i, M_2^i, \dots, M_e^i, \dots, M_h^i, \dots, M_{N_i}^i\}$, $T_{M_k^i}^{M_{k+1}^i}$ is the time required for the UAV to fly from the k th mission point to the $k + 1$ st mission in the distribution sequence, $T_0^{M_1^i}$ is the time taken by the UAV to fly from the starting point to the first mission point in the distribution sequence, $\sum_{M_k^i \in Seq^i} T_{M_k^i}^{M_{k+1}^i}$ is the time required for UAV U_i to complete all tasks in the assigned sequence, and $T_{M_{N_i}^i}^0$ is the time taken by the UAV to fly from the last mission point in the distribution sequence to the starting point. The indicator can therefore be expressed as follows:

$$f1 = \max \left(T_0^{M_1^i} + \sum_{M_k^i \in Seq^i} T_{M_k^i}^{M_{k+1}^i} + \sum_{M_k^i \in Seq^i} T_{M_k^i}^{M_{k+1}^i} + T_{M_{N_i}^i}^0 \right) \quad (5)$$

In order to ensure that the entire UAV system consumes minimal flight resources, the model proposed in this paper also considers the total flow time taken by all the UAVs to finish their assigned tasks and return to the starting point as another optimization index. This index reflects the flight resources required for the task assignment sequence to a certain extent. The shorter the index, the fewer total flight resources required for all UAVs in the entire task assignment sequence; otherwise, it indicates that the total flight resources required for the UAVs in the entire task assignment sequence are greater, and the shortest index can ensure that the UAV system consumes minimal flight resources. This indicator can be expressed as follows:

$$f2 = \sum_{i=1}^n \left(T_0^{M_1^i} + \sum_{M_k^i \in Seq^i} T_{M_k^i}^{M_{k+1}^i} + \sum_{M_k^i \in Seq^i} T_{M_k^i}^{M_{k+1}^i} + T_{M_{N_i}^i}^0 \right) \quad (6)$$

Given the dual goals of minimizing the task completion time and reducing flight resource consumption, the proposed model in this paper is designed to optimize two objective functions, $f1$ (Equation (5)) and $f2$ (Equation (6)), simultaneously. The overall flow time, defined as the total time taken by all UAVs to execute their assigned tasks and return to the command center, is intrinsically linked to the amount of flight resources consumed. A reduction in this metric generally indicates more efficient resource usage across the UAV fleet. However, minimizing the total flow time does not necessarily imply a shorter overall task duration for the UAV system. In practice, achieving a lower cumulative flow time might involve allocating a disproportionate number of tasks to specific UAVs, thereby extending the maximum completion time for individual UAVs. This trade-off highlights the inherent conflict between minimizing total flow time and minimizing the maximum individual completion time—goals respectively represented by objective functions $f1$ and $f2$. Balancing these competing objectives is thus a key consideration in the model's optimization process.

4. A Deep Reinforcement Learning-Driven Seagull Optimization Algorithm

4.1. Seagull Optimization Algorithm

Based on simulated seagull behavior, the seagull optimization algorithm is divided into two search phases: global search and local exploitation. The global search phase is carried out by simulating the migratory behavior of seagulls, and the local exploitation phase is carried out by simulating the attacking behavior of seagulls [28]. The SOA mathematically models these two behaviors and seeks the optimal solution by continuously transforming the position of the seagulls.

4.1.1. Migration Behavior

During the migration stage, each individual seagull should meet the following three conditions:

- (1) Avoid collision. The specific formula is

$$C_s = A \times P_s(t) \quad (7)$$

where C_s indicates the position where there is no collision with other seagulls, $P_s(t)$ indicates the current position of the seagull, t indicates the current iteration number, and A indicates the movement behavior of the seagull in the given search space, as shown in Equation (8):

$$A = f_c - \left(t \times \frac{f_c}{Max_{iteration}} \right) \quad (8)$$

where $Max_{iteration}$ represents the maximum number of iterations set; the hyperparameter f_c is also introduced to control the frequency of change of the variable A , and when f_c is set to 2, the variable A will decrease linearly from f_c to 0 with the number of iterations t .

- (2) Move in the direction of the optimal seagull position. This is shown in detail in Equation (9):

$$M_s = B \times (P_{best} - P_s(t)) \quad (9)$$

$$B = 2 \times A^2 \times rd \quad (10)$$

where M_s indicates the direction of the optimal seagull position, P_{best} is the current optimal seagull position, B balances global exploration with local exploitation, and rd is a random number between 0 and 1.

- (3) Constantly approach the optimal seagull position. The specific formula is

$$D_s = |C_s + M_s| \quad (11)$$

where D_s represents the distance between the individual seagull's position and the optimal seagull position.

4.1.2. Attack Behavior

Attack behavior is a local development process in which individual seagulls use a spiraling path with constantly changing angles and speeds to capture the target prey. The mathematical description in the plane represented by x , y , and z is shown in the following formulae:

$$x = r \times \cos\theta \quad (12)$$

$$y = r \times \sin\theta \quad (13)$$

$$z = r \times \theta \quad (14)$$

$$r = u \times e^{\theta v} \quad (15)$$

where r denotes the radius of the helix; θ is a random angle that falls within $[0, 2\pi]$; u and v are helical constants, where the values of u and v are taken to be 1 in the standard SOA; and e is the base of the natural logarithm.

4.2. Theoretical Framework for DRL-SOA

The DRL-SOA takes the seagull optimization algorithm as the basic framework, integrates the policy optimization mechanism of deep reinforcement learning, and adopts three DQNAgents to adaptively optimize and adjust the algorithm in the three key phases. In this framework, the seagull optimization algorithm serves as an environment for deep reinforcement learning, and the DQNAgent is introduced for training in the seagull migration, seagull attack, and further optimization phase after the attack, which prompts the DQNAgent module to optimize the candidate solution through a reward-and-punishment mechanism and adaptively adjusts the optimization strategy in each phase. In the seagull migration phase, the DQNAgent uses the search factor A as an action, as a way to achieve a balance between the global search capability and the local search capability. In the gull attack phase, an adaptive local search strategy is introduced into the action pool, which significantly improves the diversity and robustness of the algorithm in the search space. Entering the optimization phase after the attack, the DQNAgent further strengthens the optimization effect of the allocation scheme by selecting other optimization strategies. In the DRL-SOA, the strategy of action pooling is designed according to the actual problem characteristics, and the DQNAgent module is trained to adaptively adjust to the seagull migration, attack, and further optimization phases. The algorithm performs corresponding operations on the candidate solutions through the set of optimization strategies selected by the three DQNAgents and finally outputs the optimal task allocation scheme. The framework of the algorithm is shown in Figure 2, and the corresponding pseudo code is described in Algorithm 1.

Algorithm 1 DRL-SOA

Input: the parameters' configuration

Output: globally optimal individual position P_{bs}

Initialize algorithm-related parameters

Initialize the seagull population and calculate individual fitness

Set initial parameters: population size N

while $t < T$ **do**

for $i = 1$ to N

 DQNAgent1 selects the decay strategy for parameter A

 Updates on gull migration locations

 DQNAgent2 selects the local search strategy and updates the seagull individual's best attack position

 DQNAgent3 selects other strategies to further optimize and update the seagull's position

 Update the best position and fitness of the seagull individual within the current cycle

end for

end while

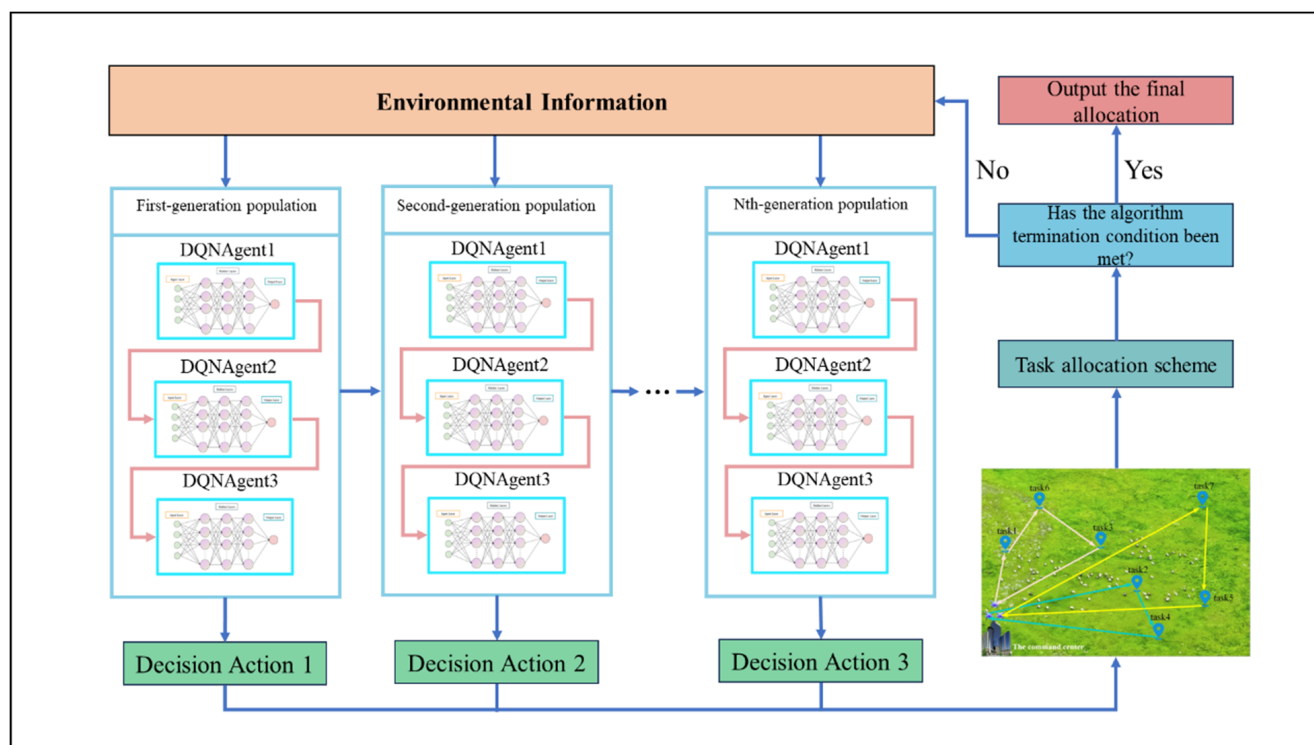


Figure 2. The DRL-SOA framework.

4.3. Deep Reinforcement Learning

Heuristic algorithms, as an emerging evolutionary computation technique, have gained widespread attention in academia and industry in recent years. Despite demonstrating significant advantages in certain problems, they often suffer from the challenge of insufficient relevance when dealing with specific problems in practical applications. Meanwhile, deep learning, as a cutting-edge technology in the field of artificial intelligence, has achieved remarkable results in several fields by virtue of its ability to speculate the probability of future events through historical data. However, the inherent static nature of deep learning methods limits their potential application in collaborative multi-UAV mission planning that requires dynamic adjustments. In contrast, reinforcement learning demonstrates unique advantages in multi-UAV cooperative mission planning by virtue of its ability to perform dynamic policy adjustment through continuous relationships between states and actions. However, due to the complexity of UAV mission planning and continuous state changes, the discretized state space is usually large and uncertain. Traditional reinforcement learning algorithms (e.g., Q-learning) have difficulty in handling such a large state space because reinforcement learning algorithms use Q-table to record the behavioral values of each operation, which greatly increases the demand for memory and computational resources when the size of the state space is too large [29]. To solve this problem, deep reinforcement learning introduces a deep neural network architecture to replace the traditional tabularized Q-value function and empirical playback techniques, thus effectively alleviating the problem of sample scarcity during training. Deep reinforcement learning not only integrates the policy optimization ability of reinforcement learning but also combines the high-level perception ability of deep learning, enabling it to perform well at complex perception and decision-making tasks, especially in dynamic and high-dimensional environments with significant advantages. Among many deep reinforcement learning algorithms, Deep Q Network (DQN) is the most widely used, so this paper introduces the DQN algorithm [30]. The key point of the DQN algorithm is that it uses an artificial neural network which replaces the Q-table, the value function of an action, with a

state in the state set and an action in the action set. The input of the network is the state information, and the output is the value of each action. The DQN algorithm is robust and can handle continuous and uncertain state set problems well. The DQN algorithm takes the observed state as input and outputs the estimated Q-values for all possible actions, thereby effectively mapping state–action pairs to their expected future rewards. The core of the DQN algorithm lies in minimizing the difference between the predicted Q-value and the target Q-value, which is calculated using a separate target network. The loss function used to train the network is defined as follows:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (16)$$

In Equation (16), the loss function is defined based on the difference between the predicted Q-value and the target Q-value. Here, θ represents the parameters (i.e., weights) of the current Q-network, which are updated during training. θ^- denotes the parameters of the target Q-network, which are periodically copied from θ to stabilize the learning process. r is the immediate reward received after executing action a in state s ; $\gamma \in [0, 1]$ is the discount factor that determines the importance of future rewards compared to immediate ones. D refers to the experience replay buffer, a memory module that stores past transition tuples (s, a, r, s') , from which mini-batches are randomly sampled to break the temporal correlations and improve training stability. The term $\max_{a'} Q(s', a'; \theta^-)$ represents the maximum estimated future Q-value for the next state s' , evaluated using the target network. The goal of training is to minimize the mean squared error between the predicted Q-value $Q(s, a; \theta)$ and the target value $r + \gamma \max_{a'} Q(s', a'; \theta^-)$.

4.3.1. State Set Design

In order to more effectively guide the DQNAgent in selecting optimization strategies, this study designed the state set based on the hypervolume (HV) metric of the population. The state set is primarily defined by the current population's HV value (HV_t), comprehensively reflecting the convergence and diversity of the solution set. Specifically, the state set is defined as follows:

$$S_t = \{HV_t\} \quad (17)$$

In this equation, HV_t denotes the hypervolume value of the population at generation t . The hypervolume metric comprehensively measures the multi-objective solution space covered by the current population, intuitively reflecting the convergence level and diversity of the solution set during optimization. An increase in the HV value indicates that the current optimization strategy positively affects the population's performance, guiding the optimization process towards an ideal state. Conversely, a decrease in HV suggests that the current strategy might negatively impact convergence or solution diversity. Thus, using HV as the main feature of the state set provides the DQNAgent with clear and effective decision-making information, enabling better guidance for strategy selection and adjustment in subsequent optimization steps.

4.3.2. Motion Space Design

For the action space design, because the algorithm proposed in this paper exists in three phases and the agents in each phase will make adaptive adjustment strategies, the action space design of the agents in each of the three phases is different. In the seagull migration phase, DQNAgent1 will adaptively select the adjustment strategy for the search factor A. The action space design in this phase includes seven strategies, such as linear descent, Sigmoid function change, nonlinear differential change, and linear differential

decrement for the search factor A . For the seagull attack stage, eight decision-making actions are designed in the action space of this stage, and the design of the decision-making actions includes a randomized local search strategy and purposive local search strategy, which can enhance the search ability of the algorithm, and the purposive local search strategy can intelligently guide the search direction of the algorithm and accelerate the convergence speed of the algorithm. The stochastic local search strategy includes a stochastic neighborhood search, stochastic variable neighborhood search, stochastic local repair search, and stochastic local taboo search. Purposeful local search strategies include adaptive neighborhood search, optimal learning search, greedy local search, and self-learning search. After the attack phase, four optimization strategies are designed in the action space of the further optimization phase to further optimize the allocation scheme. The optimization strategies in this action space include the sparrow flight mechanism, Cauchy variation, dynamic inverse learning, and adaptive t-distribution variation strategy.

4.3.3. Reward-and-Punishment Function Design

In order to more effectively guide the DQNAgent in selecting optimization strategies, we can design reward and punishment functions based on hypervolume (HV) metrics. The HV measures the volume of the objective space dominated by the current Pareto-front (PF) solution set relative to a reference point and can comprehensively reflect the convergence and diversity of the solution set. By comparing the HV of the PF solution set of the current generation (generation t) (HV_t) with that of the previous generation (generation $t - 1$) (HV_{t-1}), we can define the following reward-and-punishment mechanism:

$$R_r = \begin{cases} 2, & \text{if } HV_t > HV_{t-1} \\ 0, & \text{if } HV_t = HV_{t-1} \end{cases} \quad (18)$$

$$R_p = \begin{cases} -2, & \text{if } HV_t < HV_{t-1} \\ 0, & \text{if } HV_t = HV_{t-1} \end{cases} \quad (19)$$

4.3.4. Action Selection Strategy

In order to reduce the interdependence between the training data, the empirical playback mechanism and random sampling strategy are used in the training process. The action selection of the agent uses the Decaying- ϵ -greedy strategy, which is different from the ϵ -greedy strategy in the DQN algorithm. Specifically, the value of ϵ in the Decaying- ϵ -greedy strategy gradually decreases as the training time increases, thus effectively avoiding falling into local optimal solutions. The agent selects random actions with a probability of ϵ , and selects the action with the highest valuation in the current network with a probability of $1 - \epsilon$. The expression is as follows:

$$a_{t+1} = \begin{cases} \text{random}, & 0 < p < \epsilon \\ \text{argmax}(Q(a, s)), & \epsilon \leq p < 1 \end{cases} \quad (20)$$

where a is p random number from 0 to 1, indicating the probability of action selection.

4.3.5. The Process of DQNAgent-Adjusting the Optimization Strategies

The adjustment of the optimization strategy by the DQNAgent can be broadly divided into four steps: First, the agent collects the environmental state s , evaluates it, and calculates fitness to determine the current environmental state. Next, using the selected Decaying- ϵ -greedy strategy, the agent chooses actions a based on the values computed by the neural network. Then, based on the current state s and the action selected by the Decaying- ϵ -greedy strategy, the corresponding optimization operation is executed, changing the state

of the population to s' , and feedback is provided along with a reward value. If the reward is positive, the algorithm's action selection will become stronger; if the reward is negative, the selection will weaken. Finally, the operation s , selected action a , reward r , and state s' are stored in the memory pool, and selective DQN learning is performed. The agent then reselects and executes actions. Based on past training results, this deep reinforcement learning process is activated, thereby strengthening the action selection of the optimization strategy. The corresponding DQNAgent model is shown in Figure 3.

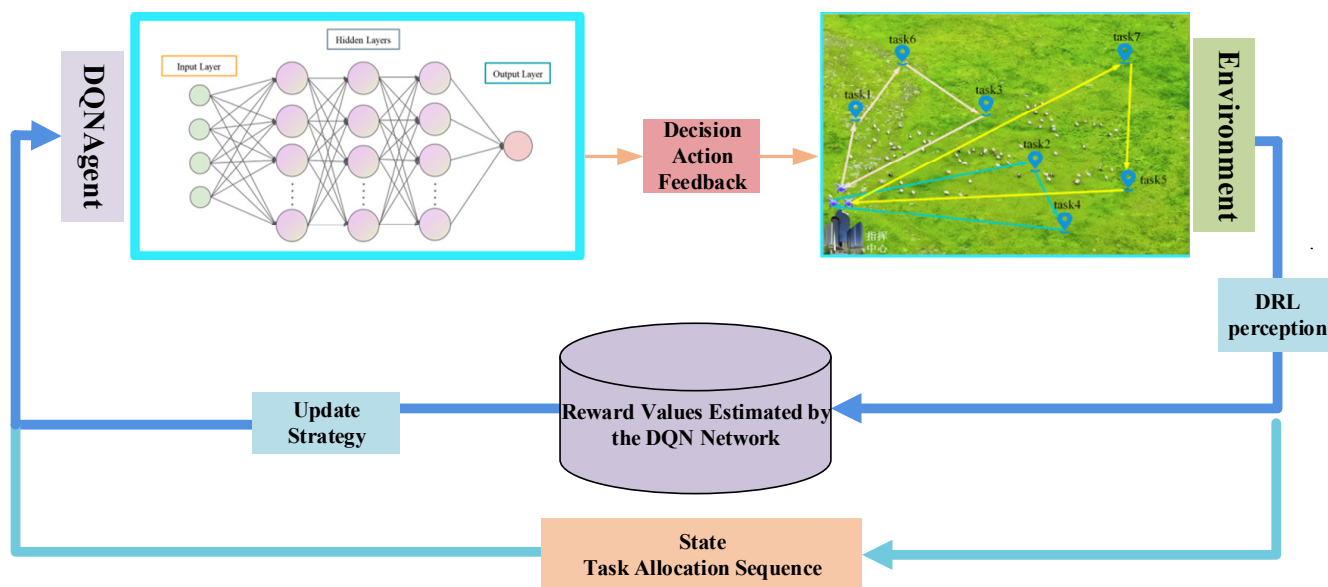


Figure 3. DQNAgent model.

5. Experiments and Analysis

5.1. Design of Experiments

To assess the performance of the proposed algorithm in addressing the multi-UAV task allocation problem, four experimental scenarios with varying task scales—comprising 20, 30, 40, and 50 tasks, respectively—were designed, with two distinct test cases under each scale. All UAVs were initialized at the coordinate origin (0, 0) and operated at a constant speed of 300 m per minute, and each agricultural task required between 3 and 5 min for execution. Details of the test cases are summarized in Table 1. The multi-UAV task allocation algorithm based on the DRL-SOA was compared with algorithms based on EMOSEA [31], INSGA-II-MTO [32], AWPSO [33], MOSOS [34], and LeCMPSO [35]. The population size of each algorithm was set to $population_size = 30$, and the maximum number of iterations was $Max_{iteration} = 500$. To reduce the impact of algorithm randomness on the experimental results, each algorithm was independently run 10 times, and the simulation environment is shown in Table 2.

Table 1. Experimental cases.

Scope of Work	Number of UAVs	Task Size	Range of Task Positions
3 km × 3 km	6	20	(100, 100) to (3000, 3000)
3 km × 3 km	6	30	(100, 100) to (3000, 3000)
4 km × 4 km	10	40	(100, 100) to (4000, 4000)
4 km × 4 km	10	50	(100, 100) to (4000, 4000)

Table 2. Simulation environment.

Item	Description
Processor	Intel®Core(TM) i5-8300H CPU @ 2.30 GHz 2.30 GHz (Intel, Santa Clara, CA, USA)
RAM	8 GB
OS	Windows 11 (64-bit)
Python version	Python 3.9

5.2. Hyperparameter Tuning

The parameters within a DRL network can be broadly categorized into model parameters and hyperparameters. Model parameters—such as the configuration of convolutional kernels and the neural network’s weight matrices—are typically learned and updated automatically through data-driven training processes. In contrast, hyperparameters must be defined manually prior to or during training. Although they are not directly influenced by the dataset, hyperparameters critically affect model behavior and performance. Examples include the learning rate and the discount factor, which govern how the model updates and values future rewards. Hyperparameters are often initialized using empirical values and then fine-tuned based on observed training outcomes. Commonly tuned hyperparameters include the greedy exploration rate ϵ , the capacity of the replay memory buffer MS , the update frequency of the Q-target network RIT , and the batch size used for sampling experiences from the replay buffer BS . For this study, these values were empirically set to $\epsilon = 0.9$, $MS = 500$, $RIT = 200$, and $BS = 32$. In scenarios employing sparse reward structures, both the learning rate and the discount factor have a direct impact on the gradient magnitude during backpropagation, thereby affecting convergence speed and stability. To systematically evaluate the sensitivity of the DRL model to these hyperparameters, a medium-scale test scenario involving 40 tasks was selected for parameter tuning. An orthogonal experimental design was employed to optimize the configuration, and the experimental setup is detailed in Table 3.

Table 3. Orthogonal experiment.

Experiment Number	α	γ
1	0.001	0.95
2	0.001	0.9
3	0.001	0.85
4	0.001	0.8
5	0.01	0.95
6	0.01	0.9
7	0.01	0.85
8	0.01	0.8
9	0.1	0.95
10	0.1	0.9
11	0.1	0.85
12	0.1	0.8

In this design, each group of experiments is trained for 4000 generations and the results of 12 groups of experiments are passed through the nondominated algorithm to obtain the nondominated solution set, which consists of 73 nondominated solutions, and the experiment numbers corresponding to the number of nondominated solutions are shown in Table 4. As can be seen from this table, the number of nondominated solutions obtained from experiment number 6 has the largest proportion, so the learning rate and the discount factor are set to $\alpha = 0.01$ and $\gamma = 0.9$, respectively.

Table 4. Parameter-tuning experimental results.

Experiment Number	Number of Non-Discretionary Solutions
1	2
2	3
3	7
4	6
5	5
6	24
7	10
8	3
9	3
10	2
11	4
12	4

5.3. Algorithm Performance Testing

To assess the effectiveness of the proposed algorithm, this study adopts two widely used performance indicators for multi-objective optimization: the hypervolume (HV) [36] and Inverted Generational Distance (IGD) [37]. The HV metric measures the quality of the solution set by calculating the volume in the objective space enclosed between the obtained non-dominated solutions and a predefined reference point. In this work, both objective functions are normalized independently, and the reference point is set at (1.2, 1.2). The HV captures both the convergence to the Pareto front and the diversity of the solution set; a higher HV value reflects a more comprehensive and better-performing set of solutions. On the other hand, IGD evaluates the proximity of the obtained solutions to the true PF. It is computed as the average Euclidean distance from each point on the reference PF to its nearest counterpart in the algorithm-generated PF. In this study, the reference Pareto front is constructed by integrating the non-dominated solution sets obtained from ten independent runs of six comparative algorithms. A lower IGD value indicates that the algorithm-generated solutions are closer to the true PF and more evenly distributed, suggesting better convergence and diversity.

Figure 4 illustrates the PF distributions obtained by the DRL-SOA and the comparison algorithms across all test cases with varying scales. As observed, the DRL-SOA consistently yields solution sets that are closer to the true Pareto front, regardless of problem size. Compared with the five comparison algorithms, the DRL-SOA demonstrates clear advantages in terms of convergence accuracy and distribution uniformity of the solution sets. In all tested scenarios, the solution sets produced by the DRL-SOA exhibit superior uniformity and better coverage of the objective space, indicating strong performance in both convergence and diversity. In contrast, the performance of the comparison algorithms is less stable. Their optimal solution sets show lower consistency in distribution and tend to deviate further from the true PF. Overall, the DRL-SOA delivers significantly better results in terms of both convergence and diversity across all test instances, reaffirming its robustness and effectiveness in multi-objective optimization tasks.

Table 5 presents the mean hypervolume (HV) values resulting from ten independent executions of each of the six algorithms on every test case across different problem scales. This metric effectively reflects both the convergence quality and diversity of the non-dominated solution sets produced by the algorithms. The highest average HV values for each scale are highlighted in bold for clarity. According to the data, the DRL-SOA consistently achieves superior average HV values compared to the other five comparison algorithms in most cases. This indicates that the DRL-SOA not only maintains better

convergence and diversity across multiple runs but also exhibits greater stability and robustness in its performance.

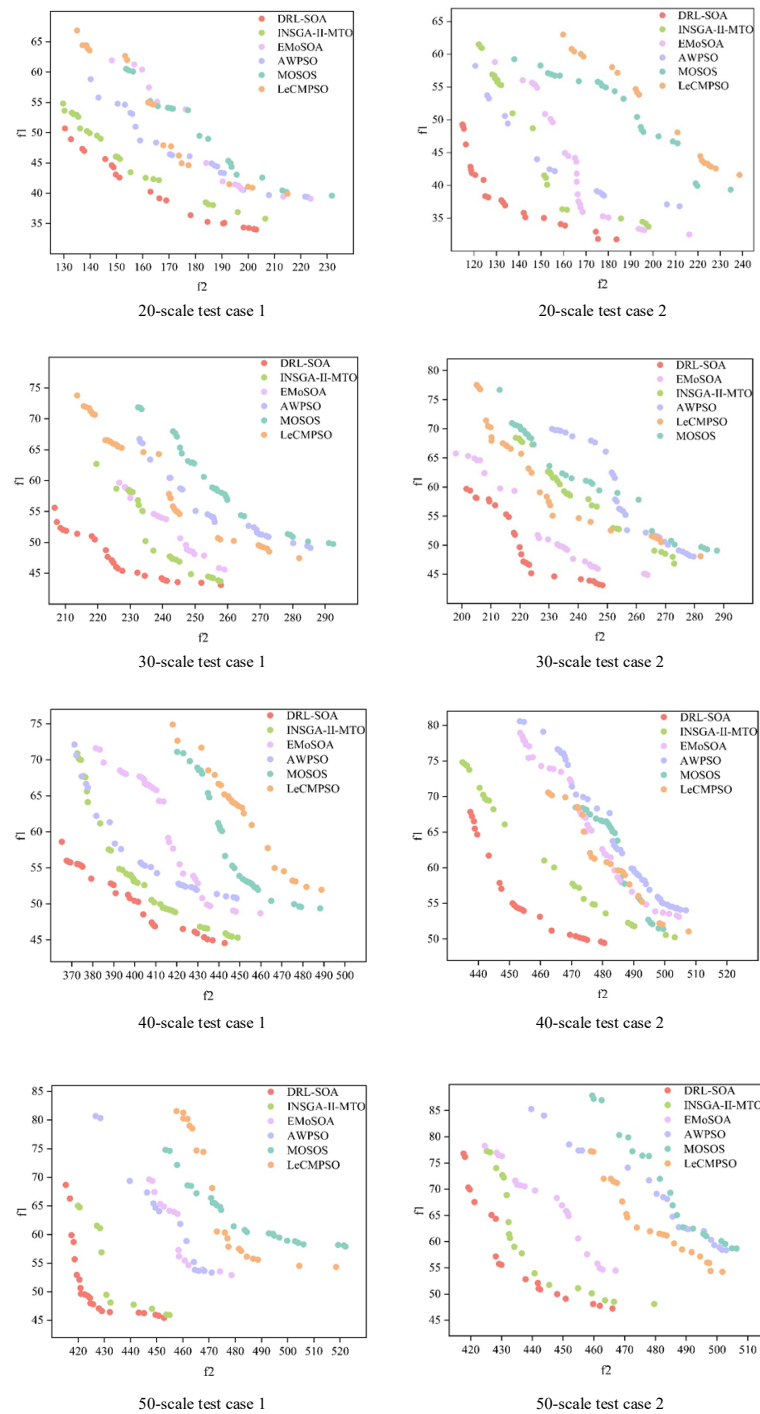
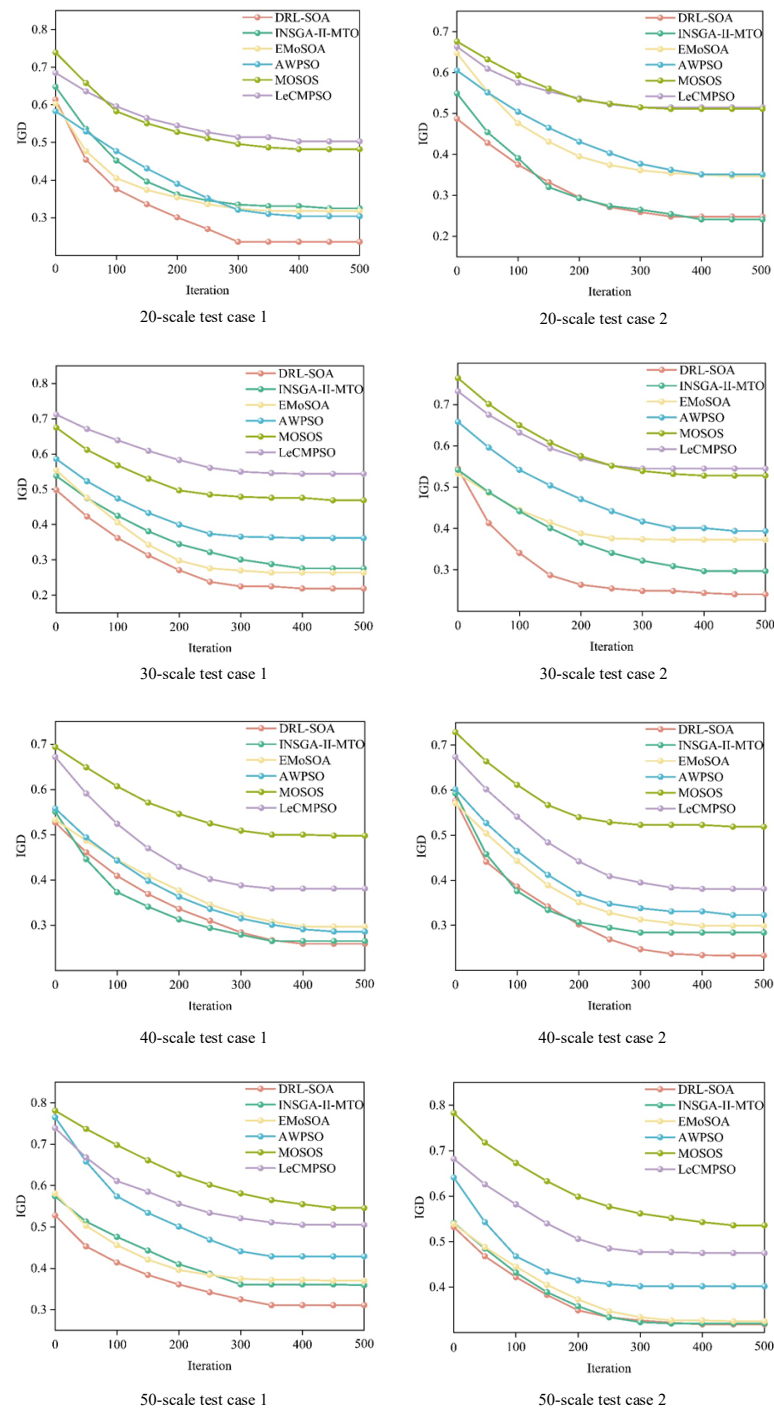


Figure 4. Comparison of non-dominated solution sets obtained by six algorithms in four scales.

Figure 5 illustrates the average Inverted Generational Distance (IGD) values recorded throughout the iterative process for all six algorithms, based on ten independent runs each. The figure clearly shows that the DRL-SOA consistently achieves lower IGD values compared to the other five comparison algorithms during most iterations. This result provides further evidence of the DRL-SOA's superior ability to produce non-dominated solution sets with enhanced convergence and diversity.

Table 5. Comparison of algorithmic average HV results.

Task Size	Test Case Number	DRL-SOA	INSGA-II-MTO	EMoSOA	AWPSO	MOSOS	LeCMPSO
20	1	1.063	0.979	0.841	0.711	0.596	0.612
	2	0.966	0.936	0.745	0.643	0.571	0.528
30	1	1.048	0.827	0.727	0.772	0.603	0.635
	2	0.955	0.903	0.886	0.804	0.678	0.674
40	1	0.931	0.968	0.739	0.785	0.692	0.659
	2	0.950	0.830	0.693	0.632	0.552	0.691
50	1	1.097	0.986	0.801	0.791	0.613	0.648
	2	0.974	0.944	0.846	0.663	0.564	0.663

**Figure 5.** Convergence curves of average IGD values at 4 scales.

The main reasons for the better performance of the DRL-SOA are as follows: Compared with traditional heuristic algorithms, the DRL-SOA introduces deep reinforcement learning, which enables the algorithm to adaptively adjust the optimization strategy. At each stage (migration, attack, and further optimization), the DQNAgent can dynamically select the optimal action based on the characteristics of the current problem, thereby improving the algorithm's adaptability in different scenarios. Second, the DRL-SOA ensures an effective balance between the global and local search during the seagull migration stage through the adaptive search factor A . This has advantages over the fixed parameter settings in traditional algorithms, enabling the algorithm to maintain high search efficiency when solving complex problems. In addition, during the seagull attack stage, the DRL-SOA greatly increases the diversity of the search space and the robustness of the algorithm by introducing a more adaptive local search strategy. This means that the algorithm can effectively avoid getting stuck in a local optimum and is therefore more likely to find a global optimum. The DRL-SOA has designed a variety of action strategies according to the needs of specific problems, including the sparrow flight mechanism, Cauchy mutation, dynamic inverse learning, and adaptive t-distribution mutation strategies. This flexible strategy design, combined with the adaptive learning ability of the DQNAgent, allows the algorithm to make optimal strategy choices at different stages, thereby achieving more efficient optimization results.

In summary, the DRL-SOA can design a specific action strategy set based on practical problems, which makes the algorithm more adaptable and efficient when dealing with specific application scenarios. In contrast, traditional heuristic algorithms usually have fixed strategies and are difficult to dynamically adjust to meet different optimization challenges. Through the collaborative work of the three DQNAgent modules, the DRL-SOA can more effectively solve MOMCTAP efficiently and ultimately achieve a better task allocation scheme.

6. Conclusions

To more accurately and objectively simulate multi-UAV operations in grassland ecological restoration tasks, this study establishes a multi-objective, multi-constraint cooperative task allocation problem (MOMCCTAP) model. The model is tailored to the practical requirements of agricultural scenarios and is designed to optimize two key objectives: (1) minimizing the maximum completion time required by any individual UAV to complete its assigned tasks and return to the origin, and (2) minimizing the total flow time of all UAVs for task execution and return. Furthermore, the model incorporates critical constraints, including multi-UAV coordination, task completion requirements, and task priority levels. To efficiently solve this complex optimization problem, a deep reinforcement learning seagull optimization algorithm (DRL-SOA) is proposed. By integrating a DQNAgent module into each phase of the seagull optimization process, the algorithm dynamically adjusts its search strategies across different stages. This adaptive mechanism enhances global exploration capabilities, refines local search precision, and promotes solution diversity, thereby improving the robustness and overall performance of the algorithm. Simulation experiments conducted on eight sets of benchmark cases demonstrate that the proposed DRL-SOA effectively balances exploration and exploitation in addressing the MOMCCTAP. The results indicate that the DRL-SOA significantly outperforms several advanced heuristic algorithms in generating high-quality task allocation schemes. Despite the excellent performance demonstrated by the DRL-SOA regarding the MOMCCTAP, there are still several research directions that deserve further exploration. First, future research could consider incorporating more kinds of optimization strategies into the algorithm's action pool to further enhance the algorithm's diversity and adaptability. Second, the current model does

not explicitly incorporate UAV energy consumption or dynamic task environments. In real-world applications, energy consumption is significantly affected by environmental factors such as altitude variation, wind resistance, and payload weight, which may constrain the feasibility of specific task assignments. Moreover, task requirements, weather conditions, and terrain may change dynamically during execution, necessitating real-time adjustments and robust decision making under uncertainty. To enhance the practicality and adaptability of the model, future research will focus on integrating accurate energy consumption modeling and extending the task allocation mechanism to support adaptive optimization in dynamic environments. These improvements are expected to further strengthen the model's applicability and robustness in complex real-world scenarios.

Author Contributions: L.Q.: responsible for experiments, data interpretation, and writing the thesis. Z.Z.: experimental data analysis, checking, experimental design. H.L.: review. Z.Y.: review. Y.D.: review, compiling technical documents, technical consultation. All authors have read and agreed to the published version of the manuscript.

Funding: The Gansu Natural Science Foundation (21JR7RA204), Gansu Natural Science Foundation (1506RJZA007), Gansu Province Higher Education Innovation Foundation (2022B-107), and Gansu Province Higher Education Innovation Foundation (2019A-056).

Data Availability Statement: The data that support the findings of this study are available upon reasonable request from the corresponding author, [Qin].

Conflicts of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Ortuani, B.; Mayer, A.; Bianchi, D.; Sona, G.; Crema, A.; Modina, D.; Bolognini, M.; Brancadoro, L.; Boschetti, M.; Facchi, A. Effectiveness of Management Zones Delineated from UAV and Sentinel-2 Data for Precision Viticulture Applications. *Remote Sens.* **2024**, *16*, 635. [\[CrossRef\]](#)
2. Kimseail; Jo, K.S.; Jin, S. Convergent Military Science and Weapon Research on the Threat of UAVs. *Korean J. Conver. Sci.* **2022**, *11*, 287–310.
3. Fatihur, R.M.F.; Shurui, F.; Yan, Z.; Lei, C. A Comparative Study on Application of Unmanned Aerial Vehicle Systems in Agriculture. *Agriculture* **2021**, *11*, 22. [\[CrossRef\]](#)
4. Jia, S.; Kai, Z.; Yang, L. Survey on Mission Planning of Multiple Unmanned Aerial Vehicles. *Aerospace* **2023**, *10*, 208. [\[CrossRef\]](#)
5. Hu, C.F.; Song, S.H.; Xu, J.J.; Wang, D.D. Distributed Task Allocation Based on Auction-PIO Algorithm for Multi-UAV Tracking. *J. Tianjin Univ. (Sci. Technol.)* **2024**, *57*, 403–414.
6. Wu, J.H.; Zhang, J.C.; Sun, Y.N.; Li, X.W.; Gao, L.J.; Han, G.J. Multi-UAV Collaborative Dynamic Task Allocation Method Based on ISOM and Attention Mechanism. *IEEE Trans. Veh. Technol.* **2024**, *73*, 6225–6235. [\[CrossRef\]](#)
7. Ma, Y.; Zhao, Y.; Bai, S.; Yang, J.; Zhang, Y. Collaborative task allocation of heterogeneous multi-UAV based on improved CBGA algorithm. In Proceedings of the 16th International Conference on Control, Automation, Robotics and Vision (ICARCV), Shenzhen, China, 13–15 December 2020.
8. Liu, X.; Jing, T.; Hou, L. An FW-GA Hybrid Algorithm Combined with Clustering for UAV Forest Fire Reconnaissance Task Assignment. *Mathematics* **2023**, *11*, 2400. [\[CrossRef\]](#)
9. Zhang, J.; Cui, Y.; Ren, J. Dynamic Mission Planning Algorithm for UAV Formation in Battlefield Environment. *IEEE Trans. Aerosp. Electron. Syst.* **2023**, *59*, 3750–3765. [\[CrossRef\]](#)
10. Peng, Q.; Wu, H.S.; Li, N.; Wang, F. A Dynamic Task Allocation Method for Unmanned Aerial Vehicle Swarm Based on Wolf Pack Labor Division Model. *IEEE Trans. Emerg. Top. Comput. Intell.* **2024**, *8*, 4075–4089. [\[CrossRef\]](#)
11. Wang, J.F.; Jia, G.W.; Lin, J.C.; Hou, Z.X. Cooperative task allocation for heterogeneous multi-UAV using multi-objective optimization algorithm. *J. Cent. South Univ. Sci. Technol. Min. Metall.* **2020**, *27*, 432–448. [\[CrossRef\]](#)
12. Zhou, Z.; Liu, H.; Dai, Y.; Qin, L. A Tent-Lévy-Based Seagull Optimization Algorithm for the Multi-UAV Collaborative Task Allocation Problem. *Appl. Sci.* **2024**, *14*, 5398. [\[CrossRef\]](#)
13. Liu, H.; Zhang, J.; Zhou, Z.; Dai, Y.; Qin, L. A Deep Reinforcement Learning-Based Algorithm for Multi-Objective Agricultural Site Selection and Logistics Optimization Problem. *Appl. Sci.* **2024**, *14*, 8479. [\[CrossRef\]](#)

14. Chen, L.Z.; Liu, W.L.; Zhong, J.H. An Efficient Multi-objective Ant Colony Optimization for Task Allocation of Heterogeneous Unmanned Aerial Vehicles. *J. Comput. Sci.* **2021**, *58*, 101545. [\[CrossRef\]](#)
15. Gao, X.H.; Wang, L.; Yu, X.Y.; Su, X.C.; Ding, Y.; Lu, C.; Peng, H.J.; Wang, X.W. Conditional probability based multi-objective cooperative task assignment for heterogeneous UAVs. *Eng. Appl. Artif. Intell.* **2023**, *123*, 106404. [\[CrossRef\]](#)
16. Huan, L.; Fuqing, Z.; Ling, W.; Jie, C.; Jianxin, T.; Jonrinaldi. An estimation of distribution algorithm with multiple intensification strategies for two-stage hybrid flow-shop scheduling problem with sequence-dependent setup time. *Appl. Intell.* **2022**, *53*, 5160–5178.
17. Xu, Y.; Sun, Z.; Xue, X.; Gu, W.; Peng, B. A hybrid algorithm based on MOSFLA and GA for multi-UAVs plant protection task assignment and sequencing optimization. *Appl. Soft Comput. J.* **2020**, *96*, 106623. [\[CrossRef\]](#)
18. Xu, Q.Z.; Ma, Y.X.; Wang, N. Task Allocation for Multi-UAV Under Dynamic Environment. *J. Nav. Aviat. Univ.* **2023**, *38*, 473–482.
19. Long, H.; Wei, C.; Duan, H.B. Task Allocation for Multi-UAV Reconnaissance via Unsupervised Learning Discrete Pigeon-Inspired Optimization. *J. Air Force Eng. Univ.* **2023**, *24*, 16–22+32.
20. Chen, C.; Liang, X.; Zhang, Z.; Zheng, K.; Liu, D.; Yu, C.; Li, W. Cooperative target allocation for air-sea heterogeneous unmanned vehicles against saturation attacks. *J. Frankl. Inst.* **2024**, *361*, 1386–1402. [\[CrossRef\]](#)
21. Liu, H.; Zhang, J.; Dai, Y.; Qin, L.; Zhi, Y. Multi-constraint distributed terminal distribution path planning for fresh agricultural products. *Appl. Intell.* **2024**, *55*, 180. [\[CrossRef\]](#)
22. Chen, H.; Xu, J.; Wu, C. Multi-UAV task assignment based on improved Wolf Pack Algorithm. In Proceedings of the 2020 International Conference on Cyberspace Innovation of Advanced Technologies, Guangzhou, China, 5 December 2020; pp. 109–115.
23. Zhang, J.D.; Chen, Y.Y.; Yang, Q.M.; Lu, Y.; Shi, G.Q.; Wang, S.; Hu, J.W. Dynamic Task Allocation of Multiple UAVs Based on Improved A-QCDPSO. *Electronics* **2022**, *11*, 1028. [\[CrossRef\]](#)
24. Yan, F.; Chu, J.; Hu, J.W.; Zhu, X.P. Cooperative task allocation with simultaneous arrival and resource constraint for multi-UAV using a genetic algorithm. *Expert Syst. Appl.* **2024**, *245*, 123023. [\[CrossRef\]](#)
25. Liu, H.; Qin, L.; Zhou, Z. Knowledge-Based Perturbation LaF-CMA-ES for Multimodal Optimization. *Appl. Sci.* **2024**, *14*, 9133. [\[CrossRef\]](#)
26. Yu, J.; Guo, J.; Zhang, X.; Zhou, C.; Xie, T.; Han, X. A Novel Tent-Levy Fireworks Algorithm for the UAV Task Allocation Problem Under Uncertain Environment. *IEEE Access* **2022**, *10*, 102373–102385. [\[CrossRef\]](#)
27. Liu, H.; Zhao, F.; Wang, L.; Xu, T.; Dong, C. Evolutionary Multitasking Memetic Algorithm for Distributed Hybrid Flow-Shop Scheduling Problem With Deterioration Effect. *IEEE Trans. Autom. Sci. Eng.* **2024**, *22*, 1390–1404. [\[CrossRef\]](#)
28. Dhiman, G.; Kumar, V. Seagull optimization algorithm: Theory and its applications for large-scale industrial engineering problems. *Knowl.-Based Syst.* **2018**, *165*, 169–196. [\[CrossRef\]](#)
29. Li, R.; Chen, S.; Xia, J.; Zhou, H.; Shen, Q.; Li, Q.; Dong, Q. Predictive modeling of deep vein thrombosis risk in hospitalized patients: A Q-learning enhanced feature selection model. *Comput. Biol. Med.* **2024**, *175*, 108447. [\[CrossRef\]](#)
30. Terven, J. Deep Reinforcement Learning: A Chronological Overview and Methods. *AI* **2025**, *6*, 46. [\[CrossRef\]](#)
31. Dhiman, G.; Singh, K.K.; Slowik, A.; Chang, V.; Yildiz, A.R.; Kaur, A.; Garg, M. EMoSOA: A new evolutionary multi-objective seagull optimization algorithm for global optimization. *Int. J. Mach. Learn. Cybern.* **2020**, *12*, 1–26. [\[CrossRef\]](#)
32. Ma, Y.; Li, B.; Huang, W.; Fan, Q. An Improved NSGA-II Based on Multi-Task Optimization for Multi-UAV Maritime Search and Rescue under Severe Weather. *J. Mar. Sci. Eng.* **2023**, *11*, 781. [\[CrossRef\]](#)
33. Deng, M.; Yao, Z.; Li, X.; Wang, H.; Nallanathan, A.; Zhang, Z. Dynamic Multi-Objective AWPSO in DT-Assisted UAV Cooperative Task Assignment. *IEEE J. Sel. Areas Commun.* **2023**, *41*, 3444–3460. [\[CrossRef\]](#)
34. Chen, H.X.; Nan, Y.; Yang, Y. Multi-UAV Reconnaissance Task Assignment for Heterogeneous Targets Based on Modified Symbiotic Organisms Search Algorithm. *Sensors* **2019**, *19*, 734. [\[CrossRef\]](#) [\[PubMed\]](#)
35. Wang, F.; Fu, Q.P.; Han, M.C.; Xing, L.N.; Wu, H.S. Learning-guided coevolution multi-objective particle swarm optimization for heterogeneous UAV cooperative multi-task reallocation problem. *Control Theory Appl.* **2024**, *41*, 1009–1017.
36. Han, H.G.; Zhang, L.L.; Yinga, A.; Qiao, J.F. Adaptive multiple selection strategy for multi-objective particle swarm optimization. *Inf. Sci.* **2023**, *624*, 235–251. [\[CrossRef\]](#)
37. Sun, Y.A.; Yen, G.G.; Yi, Z. IGD Indicator-Based Evolutionary Algorithm for Many-Objective Optimization Problems. *IEEE Trans. Evol. Comput.* **2019**, *23*, 173–187. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.